

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

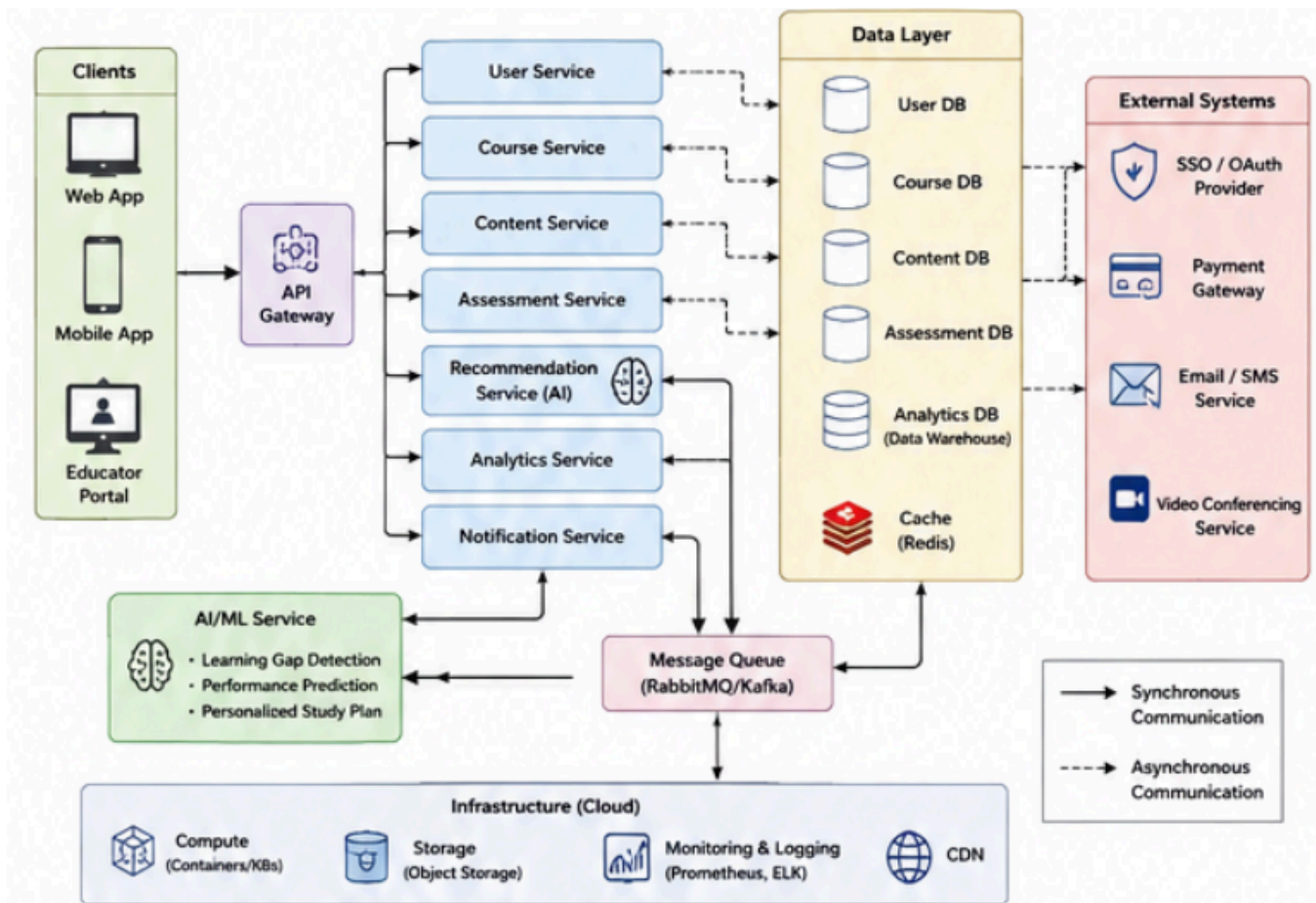
- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25



EDUVERSE: AI-POWERED ADAPTIVE LEARNING AND STUDENT SUCCESS ANALYTICS PLATFORM

1. System Requirements Analysis

1.1 System Objectives

The main objective of EduVerse is to develop an AI-powered adaptive learning platform that provides personalized education to students based on their individual learning abilities, academic performance, learning behavior, and engagement patterns. The system continuously analyzes student data and uses artificial intelligence and machine learning techniques to understand the learning needs of each student.

EduVerse aims to identify the topics in which students are facing difficulties and recommend suitable learning materials to improve their understanding. The system also predicts student academic performance and identifies students who may be academically at risk. Based on the analysis, personalized study plans can be generated for individual students.

The platform also provides educators and academic administrators with dashboards containing useful information about student performance, engagement, learning gaps, and academic

progress. This enables educators to take appropriate actions at an early stage and provide additional support to students who need it.

Overall, EduVerse aims to improve student engagement, academic performance, retention, and course completion by replacing a traditional one-size-fits-all learning approach with an intelligent and personalized learning environment.

1.2 Functional Requirements

The system should provide secure registration and login facilities for students, educators, and administrators. It should support role-based access so that each type of user can access only the functions relevant to their role.

The platform should allow educators to create and manage courses and learning materials. Students should be able to access course content such as videos, notes, assignments, quizzes, and other educational resources.

EduVerse should record student assessment results, course progress, learning activities, and engagement information. This information should be analyzed to identify learning gaps and weak areas.

The system should use machine learning algorithms to predict student performance based on historical academic results and learning behavior. It should also identify students who are potentially at risk of poor academic performance.

Based on the identified learning gaps, the recommendation system should provide personalized learning resources. The platform should also generate customized study plans according to each student's progress and learning requirements.

Educators should receive dashboards containing student performance, engagement, learning gaps, and risk information. Administrators should be able to view overall academic analytics and system-level information.

The system should also provide notifications and alerts when important actions are required, such as when a student shows signs of declining performance or requires academic intervention.

1.3 Non-Functional Requirements

EduVerse must be scalable because the number of students, courses, assessments, and learning activities can increase significantly over time. The architecture should therefore support additional users and increasing amounts of data without causing major performance problems.

The system must provide good performance because students and educators should be able to access learning content, dashboards, and recommendations without unnecessary delays.

Security is another important requirement because the platform handles personal information, academic records, assessment results, and learning analytics. Only authorized users should be able to access sensitive information.

The system should be reliable and should continue functioning even if one of its individual components experiences a temporary failure. It should also provide high availability so that students can access learning resources whenever required.

Maintainability is important because the platform may require frequent updates to its learning algorithms, recommendation mechanisms, analytics modules, and other services. The architecture should allow individual components to be modified without affecting the entire system.

The platform should also be user-friendly so that students, educators, and administrators can easily understand and use its features.

1.4 Quality Attributes

The major quality attributes influencing the EduVerse architecture are scalability, security, performance, reliability, maintainability, availability, and usability.

Scalability ensures that the platform can support a growing number of users and learning activities. Security protects student information and academic records. Performance ensures fast access to learning content and analytics. Reliability ensures that individual component failures do not affect the complete system. Maintainability allows developers to update individual components easily. Availability ensures continuous access to the learning platform, while usability ensures that the system is easy for students and educators to use.

2. Selection of Suitable Software Architecture

Selected Architecture: Microservices Architecture

The proposed EduVerse platform uses a Microservices Architecture with cloud-based infrastructure.

Microservices architecture divides the complete application into several small and independent services. Each service is responsible for a specific function of the platform. For EduVerse, services can be created for user management, course management, content management, assessments, recommendations, analytics, notifications, and AI/ML processing.

This architecture is suitable because EduVerse contains many different functions with different processing requirements. For example, the AI/ML service may require high computational resources, while the user management service may require comparatively fewer resources. Microservices allow each service to be scaled independently according to its requirements.

Another important reason for selecting microservices is maintainability. Developers can modify or update one service without making major changes to the entire application. For example, a new recommendation algorithm can be introduced in the Recommendation Service without modifying the User or Course Services.

The architecture also provides better fault isolation. If the Notification Service temporarily stops working, students can still log in, access courses, complete assessments, and view their progress.

Microservices are also suitable for integrating AI and machine learning capabilities. The AI/ML functionality can be implemented as an independent service responsible for learning-gap detection, performance prediction, personalized recommendations, and study-plan generation.

Therefore, Microservices Architecture is appropriate for EduVerse because it provides scalability, modularity, maintainability, fault isolation, and flexibility for future expansion.

3. Software Architecture

The EduVerse architecture consists of several major parts: client applications, an API Gateway, application services, AI/ML services, databases, a message queue, external systems, and cloud infrastructure.

Students can access the platform through a web or mobile application. Educators and administrators can use their respective dashboards to monitor academic information and manage the platform.

All requests from the clients are first received by the API Gateway. The API Gateway acts as a single entry point to the backend services. It performs authentication, request routing, security checks, and rate limiting before forwarding requests to the appropriate service.

The backend contains several independent services. The User Service manages user accounts, profiles, authentication, and roles. The Course Service manages courses and enrollment information. The Content Service manages learning materials. The Assessment Service manages quizzes, assignments, examinations, and results.

The Recommendation Service generates personalized learning-resource recommendations. The Analytics Service processes student data and generates performance and engagement analytics. The Notification Service sends alerts and reminders to students and educators.

An independent AI/ML Service performs intelligent operations such as learning-gap detection and student performance prediction. It receives relevant student data and provides predictions or recommendations to other services.

The Data Layer stores user information, course information, assessment results, learning activity, and analytics data. A caching system can be used to store frequently accessed information and improve response time.

A message queue is used for asynchronous communication between services. This is particularly useful for analytics, AI processing, notifications, and other background operations.

The complete system can be deployed on cloud infrastructure that provides computing resources, storage, monitoring, logging, networking, and other required services.

4. Components and Their Interactions

The Client Applications provide the interface through which students, educators, and administrators interact with EduVerse.

The API Gateway receives client requests and routes them to the appropriate backend service. It also handles authentication, authorization, and security-related functions.

The User Service manages user accounts and roles. It determines whether a user is a student, educator, or administrator and provides access according to the assigned role.

The Course Service manages course information, enrollment, and course-related operations. The Content Service manages educational resources such as videos, documents, notes, and other learning materials.

The Assessment Service manages quizzes, assignments, examinations, submissions, and scores. Assessment results are important inputs for the analytics and AI/ML components.

The AI/ML Service analyzes student information to identify learning gaps and predict academic performance. It can use assessment scores, learning activity, engagement, and previous performance as input.

The Recommendation Service uses the results generated by the AI/ML system to recommend appropriate learning resources. It can recommend additional explanations, practice exercises, quizzes, or other learning materials.

The Analytics Service processes student and course data to generate dashboards and reports for educators and administrators.

The Notification Service sends important alerts and reminders through appropriate communication channels.

The Database Layer stores the information required by different services. Separate data stores can be maintained for different types of information to improve organization and scalability.

The Message Queue allows services to communicate asynchronously. This reduces unnecessary dependency between services and allows background tasks to be processed without blocking the main application.

5. Overall System Workflow

The EduVerse workflow begins when a student logs into the platform through the web or mobile application.

The request is sent to the API Gateway, which verifies the user's identity and forwards the request to the appropriate service.

The student then accesses a course and interacts with learning materials such as videos, notes, assignments, and quizzes. The system records the student's learning activities, assessment results, progress, and engagement information.

This information is sent to the Analytics and AI/ML components for processing. The AI/ML system analyzes the student's learning behavior and academic performance to identify areas where the student requires improvement.

For example, if a student consistently performs poorly in a particular topic, the system can identify that topic as a learning gap. The Recommendation Service can then recommend additional learning materials, practice questions, or revision content related to that topic.

The AI/ML system can also predict the student's future academic performance. If the prediction indicates that the student is at risk of poor performance, the system can notify the educator and generate an appropriate intervention.

The student's personalized study plan can then be updated according to their current progress and learning requirements.

Educators can view this information through their dashboard. They can monitor individual students, identify learning difficulties, observe performance trends, and provide appropriate academic support.

6. Quality Attribute Analysis

6.1 Scalability

The microservices architecture allows each service to be scaled independently. For example, if a large number of students access learning content at the same time, the Content Service can be scaled without unnecessarily scaling the User or Notification Services.

Cloud infrastructure can also provide additional computing resources when the workload increases.

6.2 Security

Security is implemented using authentication, authorization, encrypted communication, secure APIs, and role-based access control.

Students should only be allowed to access their own academic information. Educators should have access to information about the students they are responsible for, while administrators can have broader system-level access.

Sensitive student information should be protected both during transmission and while stored.

6.3 Performance

Performance can be improved through caching, optimized databases, content delivery networks, efficient APIs, and asynchronous processing.

Frequently accessed course information can be stored in a cache so that it can be retrieved faster. Long-running AI and analytics operations can be processed in the background using a message queue.

6.4 Reliability

Microservices improve reliability because a failure in one service does not necessarily cause the entire system to fail.

For example, if the Notification Service becomes temporarily unavailable, students can still access courses and complete assessments.

Service monitoring, health checks, backup mechanisms, and fault recovery can further improve reliability.

6.5 Maintainability

Each microservice has a specific responsibility. This makes the system easier to understand, test, modify, and maintain.

For example, developers can update the AI/ML model without changing the Course Service or User Service.

6.6 Availability

The platform can achieve high availability by deploying multiple instances of important services and using cloud infrastructure, load balancing, monitoring, automatic recovery, and database backup mechanisms.

This allows students and educators to continue using EduVerse even when individual components experience problems.

7. Architectural Justification

The Microservices Architecture was selected because it matches the different functional and technical requirements of EduVerse.

The platform requires continuous processing of student data, personalized recommendations, machine learning predictions, analytics, and content delivery. These functions have different resource requirements, so implementing them as independent services provides better flexibility.

The AI/ML component is separated from the main application because machine learning models may require specialized processing resources and frequent updates. Independent deployment allows the AI system to evolve without affecting other services.

A message queue is introduced to support asynchronous processing. This is useful for activities such as collecting learning events, processing analytics, generating predictions, and sending notifications.

Cloud infrastructure is selected because the platform needs to support varying workloads. During examination periods or other high-traffic situations, additional resources can be provided to maintain system performance.

The architecture also supports integration with external systems such as existing learning management systems, authentication services, email or SMS services, and video conferencing platforms.

8. Architectural Trade-off

Although microservices provide scalability and flexibility, they also introduce additional complexity compared with a monolithic architecture.

A microservices-based system requires more services to be deployed, monitored, secured, and maintained. Communication between services also introduces network overhead.

Using multiple databases can improve service independence but can make data consistency and management more complicated.

Similarly, introducing a message queue improves asynchronous processing and reliability but requires additional infrastructure and monitoring.

Despite these disadvantages, the benefits are more suitable for EduVerse because the platform is expected to handle large amounts of student data, AI processing, analytics, and personalized learning activities.

Therefore, the additional complexity of microservices is justified by the scalability, flexibility, maintainability, and future expansion requirements of the platform.

9. Future Enhancements

The architecture can be extended in the future by adding new independent services.

An AI Tutor Service could provide students with conversational academic assistance. A Gamification Service could introduce points, badges, achievements, and leaderboards to improve engagement.

A Plagiarism Detection Service could analyze student assignments. A Voice Learning Service could provide voice-based interaction with educational content.

The platform could also integrate with external learning management systems and provide additional dashboards for parents or academic administrators where appropriate.

Because the system uses independent services, these new capabilities can be introduced without completely redesigning the existing platform.

10. Conclusion

EduVerse is designed as an AI-powered adaptive learning and student success analytics platform that addresses the limitations of traditional one-size-fits-all education.

The proposed Microservices Architecture provides a suitable foundation for the system because it supports independent development, scalability, maintainability, reliability, and integration with AI/ML technologies.

The architecture enables EduVerse to collect student learning data, analyze performance and engagement, identify learning gaps, recommend personalized resources, predict academic outcomes, and provide actionable information to educators.

By combining microservices, AI/ML processing, analytics, asynchronous communication, databases, and cloud infrastructure, EduVerse can provide personalized learning experiences while helping educators identify and support academically at-risk students.

The architecture can also be extended in the future with AI tutors, gamification, advanced analytics, plagiarism detection, voice-based learning, and other educational technologies.